FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Iago Amorim Lebron de Lima

Produto: **CuidaHome**
https://github.com/Lebron057/saude-domiciliar

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 3
3.2 Onde roda 4
3.3 Fluxo de uma requisição 5
3.4 Fluxo de assinatura 6
3.5 Modelo de dados do domínio 7
3.6 Segurança 8
3.7 Observabilidade, CI/CD e documentação da API 9
3.8 Camadas IaaS, PaaS e SaaS do projeto 9
4 Quadro de entregas 10
5 Atividades realizadas 11
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 11
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 13
5.3 Atividade 9 — Fork e plano de negócio 15
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 16
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 18
6 Considerações finais 20
7 Declaração de autoria 20

1 Introdução

A disciplina de **Introdução à Computação em Nuvem (ISW055)**, ministrada pelo Prof. Allan Lincoln Rodrigues Siriani na Fatec Pompeia durante o semestre 2026.2, propõe uma abordagem formativa prática centrada no modelo de responsabilidade compartilhada, conteinerização de aplicações, arquitetura de microsserviços, isolamento lógico de inquilinos (**multi-tenancy**) e publicação em ambientes reais de produção. Em vez de limitar o aprendizado a abstrações comerciais ou simulações locais isoladas, a disciplina construiu um ecossistema que atua como um provedor de nuvem em miniatura, combinando servidores **on-premises** virtualizados com nós de borda em nuvem pública permanente (**GCP Always Free**) para contorno de limitações severas de infraestrutura, como o CGNAT.

Ao longo do semestre, a trajetória prática transitou por duas fases evolutivas complementares. No 1º bimestre (P1), os estudantes construíram uma aplicação acadêmica em camadas voltada para o catálogo de filmes do ator Tom Hanks consumindo a API pública do TMDB. Esse projeto foi submetido a sucessivas refatorações: nasceu como um monólito conteinerizado com segregação de inquilinos em nível de código (Atividade 2), teve sua camada de identidade desacoplada em um microsserviço isolado em rede interna Docker (Atividade 3) e consolidou uma governança robusta com controle de acesso baseado em papéis (**RBAC**) com **enforcement** centralizado de backend (Atividade 4).

No 2º bimestre, esse arcabouço técnico serviu de fundação para o desenvolvimento de um produto comercial de verdade (**SaaS**). Através de um processo formal de **fork** e migração de histórico, a aplicação transcendeu o nicho cinematográfico para conceber o **CuidaHome** (**Saúde & Atendimento Domiciliar**). A plataforma resolve uma dor latente enfrentada por famílias em situações de pós-alta cirúrgica ou traumas ortopédicos súbitos: a dificuldade crítica de encontrar cuidadores e enfermeiros certificados de confiança em prazos emergenciais e a burocracia arcaica (exigência de fiador, notas promissórias e cauções presenciais) para locação de camas hospitalares e cadeiras de rodas por curtos períodos (60 a 90 dias).

O presente relatório técnico-consolidado documenta integralmente o estado final do sistema em produção e presta contas das atividades desenvolvidas no 2º bimestre. O documento está estruturado em seis macroetapas: a Seção 2 apresenta a metodologia de auditoria e fontes de dados; a Seção 3 detalha minuciosamente a arquitetura técnica do SaaS em execução no domínio público; a Seção 4 traz o quadro sinóptico de entregas gerado a partir das fichas individuais; a Seção 5 examina as atividades realizadas da 7 à 11 com links clicáveis e evidências visuais comprobatórias; e a Seção 6 encerra com as considerações finais e lições aprendidas em computação em nuvem.

2 Metodologia

As atividades avaliativas que compõem este relatório bimestral foram executadas seguindo um processo rigoroso de versionamento contínuo e rastreabilidade documental. As atividades 7 (Plano Premium com Stripe) e 8 (Documentação Reversa e Arquitetura Cloud) foram implementadas originalmente no repositório de origem do catálogo de filmes (`https://github.com/Lebron057/tom-hanks-avestruz`), o mesmo utilizado nas entregas do 1º bimestre.

Na Atividade 9, em observância às diretrizes pedagógicas da disciplina, o repositório original foi clonado com seu histórico integral preservado (`git clone --bare` seguido de `git push --mirror`) para um novo repositório público intitulado `https://github.com/Lebron057/saude-domiciliar`. Nesse novo repositório, toda a camada funcional de filmes foi arquivada e transposta semanticamente para as entidades do domínio de saúde, onde foram desenvolvidas e homologadas as atividades 10 (domínio no ar com micros-

serviços, 4 papéis no RBAC, Redis Streams e MinIO) e 11 (Stripe Billing recorrente, cauções patrimoniais, portal do cliente e aplicação de quotas na API).

Para garantir a confiabilidade das evidências, a data e a hora exatas de cada entrega foram auditadas diretamente a partir do histórico de commits do Git através do comando:

```
git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad %s"
```

As informações temporais e os identificadores de commit foram cruzados com as páginas oficiais dos commits na interface web do GitHub e, a partir da atividade 9, com a issue de entrega oficial aberta no repositório com menção ao docente `\@siriani`. O sistema foi desenvolvido e testado localmente em ambiente Docker Compose e publicado em produção no cluster da Fatec gerenciado via Portainer CE sob o subdomínio determinístico `https://iago-lima-isw055.lapps.studio`, conectado à porta pública reservada 8210 através de um túnel reverso sustentado por uma instância `e2-micro` Always Free no Google Cloud Platform.

3 Arquitetura do SaaS

Esta seção descreve a especificação técnica e operacional da plataforma **CuidaHome** que se encontra em efetivo funcionamento em produção no subdomínio atribuído, detalhando microsserviços, fluxos de dados, modelos relacionais e políticas de segurança.

3.1 Visão geral e componentes

A arquitetura do **CuidaHome** é baseada em microsserviços conteinerizados orquestrados através do Docker Compose no Portainer. A aplicação implementa o princípio do menor privilégio na camada de rede: apenas a borda pública (`catalog-service`) expõe porta para o host através da porta reservada 8210. Todos os demais componentes residem estritamente na rede privada interna do Docker (`internal-net`), comunicando-se exclusivamente via resolução nativa de nomes DNS do daemon.

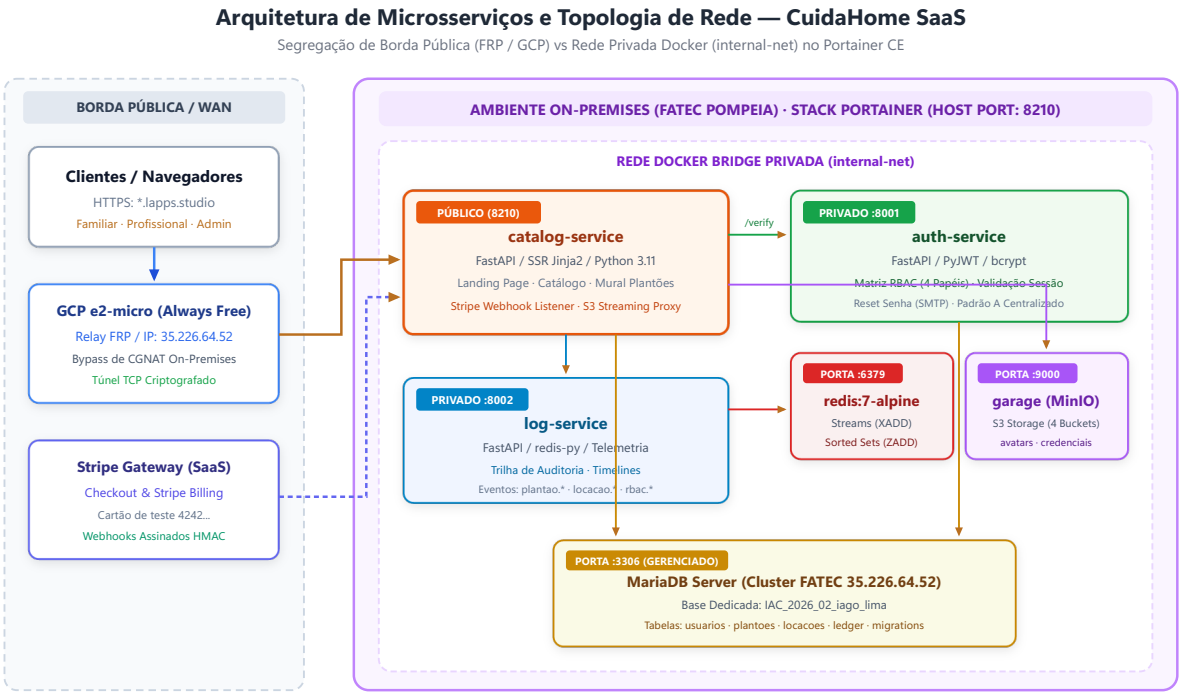


Figura 1: Diagrama de componentes e microsserviços do SaaS CuidaHome

Componente	Responsabilidade	Tecnologia / Imagem	Público?
catalog-service	Gateway de borda, SSR com Jinja2, catálogo ortopédico, mural de plantões, proxy S3 e listener de webhooks	lebron057/catalog--service (Python 3.11 / FastAPI)	Sim (Porta 8210 via FRP)
auth-service	Cadastro qualificado, login, emissão/ validação de JWT, matriz RBAC (4 papéis) e reset de senha via SMTP	lebron057/auth--service (FastAPI / PyJWT / bcrypt)	Não (Rede interna :8001)
log-service	Trilha de auditoria centralizada, recepção de eventos de negócio e contadores de telemetria em tempo real	lebron057/log--service (FastAPI / redis-py)	Não (Rede interna :8002)
redis	Barramento assíncrono de eventos via Redis Streams (`evt:*`) e cronologia de entidades via Sorted Sets	redis:7-alpine	Não (Rede interna :6379)
MariaDB	Persistência relacional do domínio, tabelas de saúde, assinaturas, controle de migrações e ledger financeiro	MariaDB Server FATEC (Cluster 35.226.64.52)	Não (Cluster :3306)
garage (MinIO)	Armazenamento de objetos compatível com API S3 particionado em 4 buckets (`avatars`, `equipamentos`, `credenciais`, `documentos`)	minio/ minio:RELEASE.2023-09-07T02-05-19Z	Não (Rede interna :9000)
Stripe (externo)	Checkout de diárias de plantões, assinaturas recorrentes de locação (Billing), caução hold e portal do cliente	Gateway SaaS Externo	Sim (Webhook → catalog--service)

3.2 Onde roda

O **CuidaHome** opera na mesma infraestrutura **multi-tenant** fornecida para a disciplina desde o início do semestre. O sistema está implantado como uma Stack gerenciada no painel **Portainer Community Edition** (`https://portainer.lapps.studio`), sob o usuário atribuído `IAC_2026_02_iago_lima`. O código--fonte em execução é puxado do repositório novo (`https://github.com/Lebron057/saude-domiciliar`).

A máquina servidora local da Fatec opera atrás de **CGNAT (**Carrier-Grade NAT**)**, o que inviabiliza conexões diretas de entrada pela internet pública. Para contornar essa restrição sem onerar o projeto, o tráfego de entrada é capturado por uma máquina virtual `e2-micro` provisionada na camada permanente gratuita (**Always Free**) do Google Cloud Platform (GCP) com o IP público `35.226.64.52`. Essa instância executa um servidor **FRP (**Fast Reverse Proxy**)**, que mantém um túnel TCP reverso e criptografado direto com o Docker Engine local, roteando o tráfego do subdomínio público `https://iago-lima-isw055.lapps.studio` exclusivamente para a porta reservada **8210** do host.

A fim de assegurar a estabilidade do cluster compartilhado entre os 30 alunos, os contêineres operam sob limites formais de recursos configurados no Portainer:

- `catalog-service`: limite de 512 MB de memória RAM e cota de 0.5 CPU;
- `auth-service`: limite de 256 MB de memória RAM e cota de 0.25 CPU;
- `log-service`: limite de 256 MB de memória RAM e cota de 0.25 CPU;
- `garage` (MinIO): limite de 512 MB de memória RAM e cota de 0.5 CPU;

- `redis`: limite de 128 MB de memória RAM e cota de 0.2 CPU.

As configurações e segredos são fornecidos exclusivamente através de variáveis de ambiente gerenciadas na aba **Environment Variables** da Stack do Portainer, nunca expostas no repositório. As variáveis operacionais configuradas são: `APP_NAME`, `PUBLIC_BASE_URL`, `PLATFORM_FEE_PERCENT`, `APP_PORT`, `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `SECRET_KEY`, `AUTH_SERVICE_URL`, `LOG_SERVICE_URL`, `MINIO_ENDPOINT`, `MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`, `MINIO_BUCKET`, `S3_BUCKET_AVATARS`, `S3_BUCKET_EQUIPAMENTOS`, `S3_BUCKET_CREDENCIAIS`, `S3_BUCKET_DOCUMENTOS`, `STRIPE_API_KEY`, `STRIPE_WEBHOOK_SECRET`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD` e `SMTP_FROM`.

3.3 Fluxo de uma requisição

Para ilustrar o ciclo de comunicação entre os microsserviços, tomemos como referência a ação transacional central da plataforma: **um familiar de paciente solicita a contratação de um plantão de cuidador**.

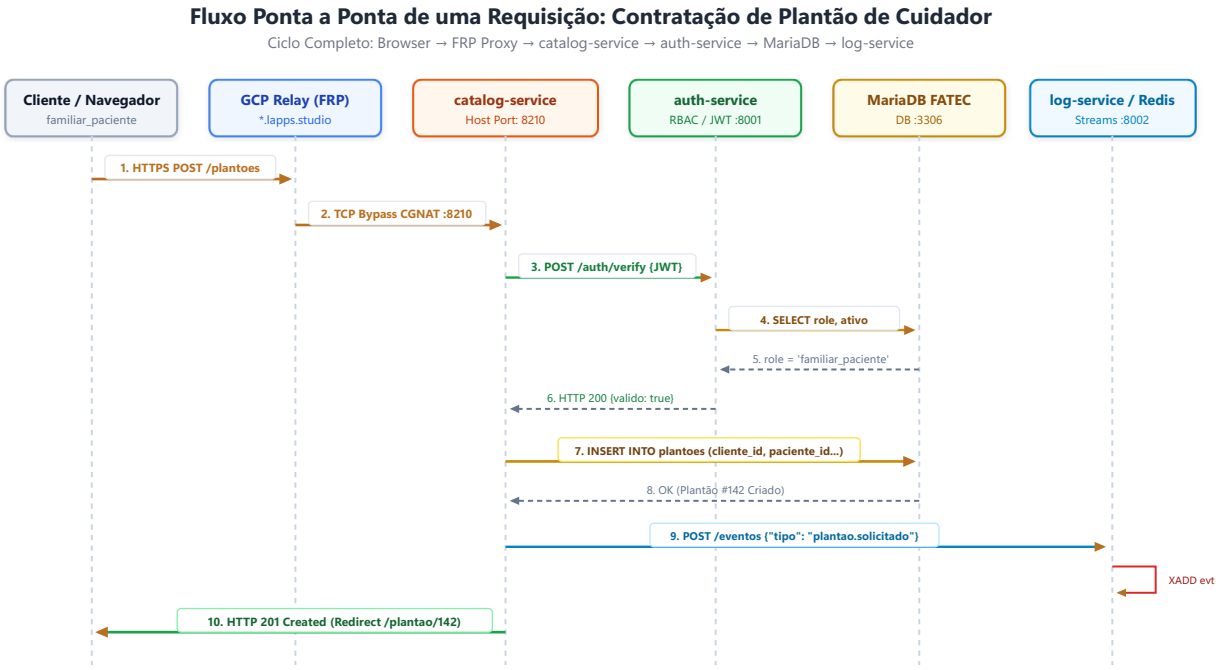


Figura 2: Fluxo ponta a ponta de uma requisição: solicitação de plantão de atendimento domiciliar

O fluxo obedece rigorosamente às seguintes etapas:

1. **Requisição do Cliente:** O navegador do usuário familiar submete um formulário via `POST /plantoes/solicitar` transmitindo os dados do atendimento e seu cookie de sessão seguro (`access_token`).
2. **Encaminhamento de Borda:** A requisição chega à máquina de borda no GCP (`*.lapps.studio`), que através do túnel reverso FRP repassa os pacotes TCP diretamente para a porta 8210 do contêiner `catalog-service`.
3. **Validação de Identidade (Padrão A):** O `catalog-service` intercepta o cookie e efetua uma chamada interna síncrona `POST /auth/verify` para o `auth-service` na rede privada.
4. **Consulta de Governança no Banco:** O `auth-service` decodifica o JWT e consulta a tabela `usuarios` no MariaDB para atestar se a conta está ativa e se possui o papel exigido (`role = 'familiar_paciente'`).
5. **Autorização Confirmada:** O `auth-service` responde com `HTTP 200 OK` atestando a identidade do solicitante e seu identificador primário.

6. **Gravação Transacional com Isolamento:** O `catalog-service` monta a instrução SQL parametrizada inserindo o novo registro na tabela `plantoes`, vinculando obrigatoriamente a chave estrangeira `cliente_id = session.user_id` para impedir qualquer brecha de ataque IDOR.
7. **Emissão de Evento de Auditoria:** O `catalog-service` despacha assincronamente (**best-effort**) uma notificação HTTP `POST /eventos` para o `log-service` informando `tipo = "plantao.solicitado"` e o identificador da entidade.
8. **Persistência em Streams:** O `log-service` grava o payload no Redis Stream `evt:plantao` via comando `XADD` e atualiza a linha do tempo do paciente em Sorted Set.
9. **Retorno ao Cliente:** O `catalog-service` retorna ao navegador a resposta `HTTP 201 Created` redirecionando o familiar para o fechamento do pagamento da diária.

3.4 Fluxo de assinatura

A monetização de locação ortopédica (camas hospitalares motorizadas e cadeiras de rodas) é sustentada pela infraestrutura do **Stripe Billing** operando em modo de assinatura recorrente (`mode=subscription`):

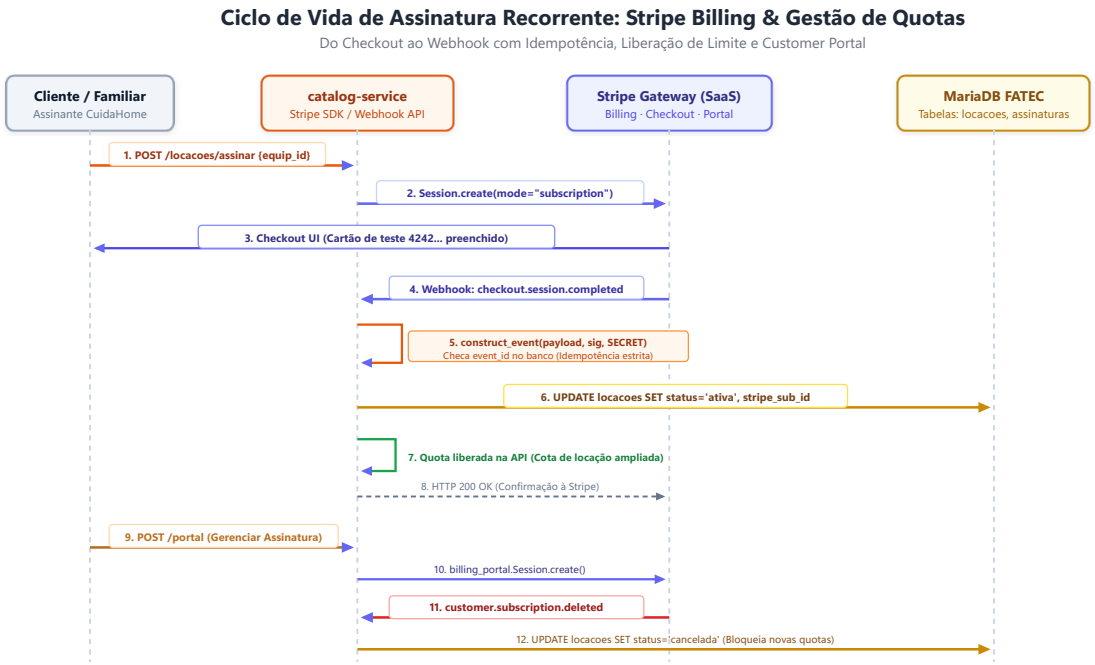


Figura 3: Ciclo de vida da assinatura: checkout ⟶ webhook ⟶ plano no banco

1. **Início do Checkout:** Ao selecionar um equipamento no catálogo (ex.: Cama Hospitalar por R$ 290,00/mês), o cliente clica em "Assinar Locação". O backend invoca a API do Stripe (`stripe.checkout.Session.create`) configurada com `mode="subscription"`, associando o identificador de preço recorrente (`stripe_price_id`), o ID do cliente e parâmetros de caução patrimonial.
2. **Pagamento no Stripe:** O cliente é redirecionado para o ambiente seguro do Stripe Checkout e insere os dados do cartão de teste oficial `4242 4242 4242 4242`. A transação de assinatura mensal é autorizada e a garantia de caução (R$ 500,00) é retida via pré-autorização (`capture_method=manual`).
3. **Disparo do Webhook:** O gateway do Stripe emite os eventos transacionais `checkout.session.completed` e `customer.subscription.created` direcionados ao endpoint público `/webhook/stripe` da aplicação.
4. **Validação Criptográfica e Idempotência:** O `catalog-service` intercepta a requisição e valida a assinatura criptográfica presente no cabeçalho `Stripe-Signature` utilizando a chave secreta

compartilhada (`STRIPE_WEBHOOK_SECRET`) através de `stripe.Webhook.construct_event`. Em seguida, inspeciona a tabela `processamento_webhooks` no MariaDB: se o identificador do evento (`event.id`) já constar como gravado, o backend interrompe o fluxo e retorna imediatamente `HTTP 200 OK`, evitando duplicação de faturamento ou ativação duplicada caso a Stripe reenvie o evento.

5. **Ativação da Locação no Banco:** Confirmada a novidade do evento, a aplicação executa a atualização no MariaDB registrando a nova locação na tabela `locacoes` com `status = 'ativa'`, armazena o `stripe_subscription_id` e associa a data de renovação.
6. **Enforcement de Quotas na API:** A cada requisição de locação de equipamento, a API valida a quantidade de assinaturas vigentes do cliente no banco. Se o usuário atingir o limite máximo permitido pelo seu plano atual (1 equipamento simultâneo no plano básico), a API bloqueia novas tentativas retornando **HTTP 403 Forbidden**, exigindo upgrade para liberação de cotas adicionais.
7. **Cancelamento via Customer Portal:** O usuário tem acesso ao botão "Gerenciar Assinatura" em seu painel, que gera uma sessão segura no Stripe Customer Portal (`stripe.billing_portal.Session.create`). Quando o cliente cancela a renovação no portal, a Stripe dispara o webhook `customer.subscription.deleted`, que atualiza a locação no MariaDB para `status = 'cancelada'`, revoga as cotas ativas e programa o recolhimento do equipamento.

3.5 Modelo de dados do domínio

O modelo de persistência do **CuidaHome** foi construído sobre o banco de dados relacional MariaDB (`IAC_2026_02_iago_lima`) através de um motor determinístico de migrações versionadas (`db/migrate.py`), estruturado em tabelas com chaves estrangeiras, índices e restrições de unicidade:

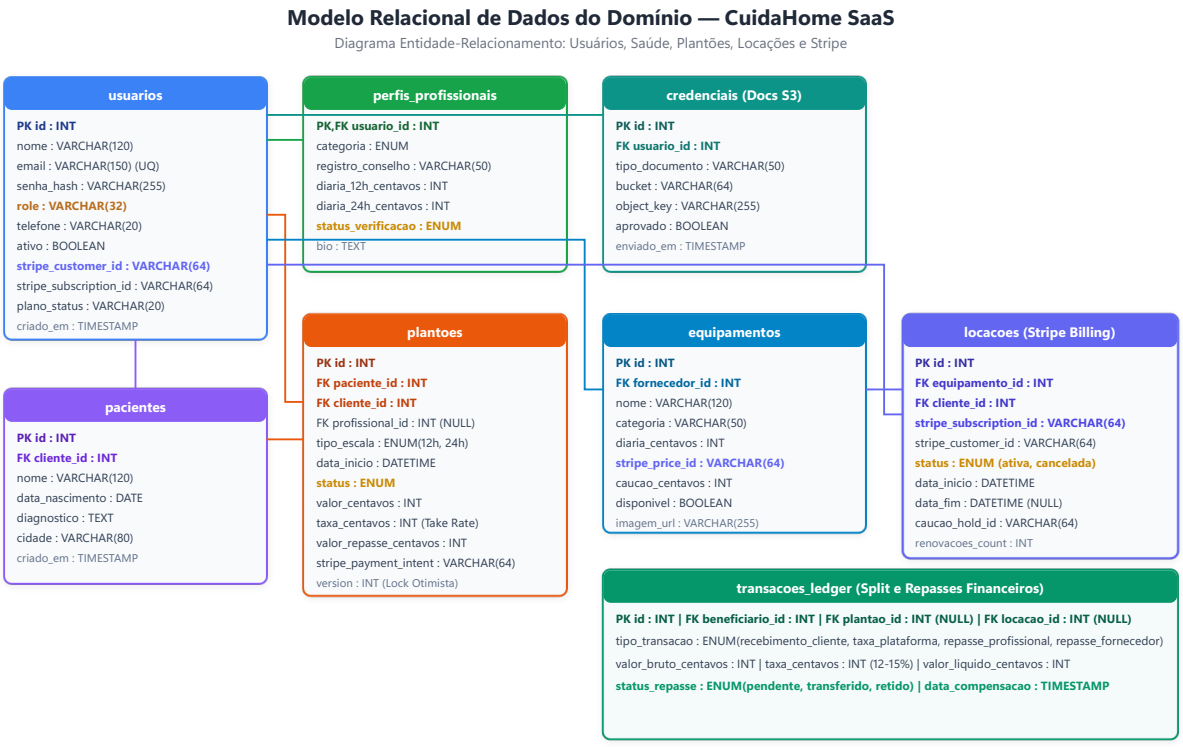


Figura 4: Modelo de dados relacional do domínio CuidaHome

As principais entidades e seus relacionamentos compreendem:

- **`usuarios`:** Tabela central de identidades armazenando `id` (PK), `nome`, `email` (UNIQUE), `senha_hash` (bcrypt), `role` (matriz de 4 papéis), `telefone`, `ativo`, `stripe_customer_id` e data de cadastro.

- **`perfis_profissionais`:** Extensão 1:1 de usuários cadastrados como prestadores de saúde, com `usuario_id` (PK, FK), categoria profissional (`cuidador`, `tecnico_enfermagem`, `enfermeiro`), número do registro no conselho de classe (COREN), valor das diárias de 12h e 24h em centavos e status de homologação (`pendente`, `aprovado`, `rejeitado`).
- **`credenciais`:** Armazena metadados dos diplomas e certidões salvos no MinIO S3, com `id` (PK), `usuario_id` (FK), `bucket`, `object_key`, `tipo_documento` e flag booleana de validação documental.
- **`pacientes`:** Cadastro de dependentes ou familiares sob tutela de um cliente, contendo `id` (PK), `cliente_id` (FK `usuarios.id`), nome do paciente, diagnóstico resumido e cidade.
- **`plantoes`:** Escalas de atendimento domiciliar diretas ou abertas no mural, contendo `id` (PK), `paciente_id` (FK), `cliente_id` (FK), `profissional_id` (FK, nulo enquanto aberto no mural), status do ciclo de vida, valor da diária, taxa de intermediação retida pela plataforma (`taxa_centavos`), valor líquido de repasse e coluna `version` para controle de concorrência com lock otimista.
- **`equipamentos`:** Catálogo de aparelhos hospitalares (camas motorizadas, cadeiras de rodas, muletas), contendo `id` (PK), `fornecedor_id` (FK `usuarios.id`), nome, categoria, diária base, identificador do produto na Stripe (`stripe_price_id`) e valor da caução em centavos.
- **`locacoes`:** Contratos de locação recorrente sustentados pelo Stripe Billing, contendo `id` (PK), `equipamento_id` (FK), `cliente_id` (FK), `stripe_subscription_id` (chave de integração com a Stripe), `status` (`ativa`, `encerrada`, `inadimplente`), `data_inicio`, `data_fim` e identificador da retenção de caução (`caucao_hold_id`).
- **`transacoes_ledger`:** Livro-razão financeiro que registra o split de pagamentos e a retenção de taxa da plataforma (12% a 15%) sobre cada transação, viabilizando o extrato contábil dos repasses.

3.6 Segurança

A segurança e governança de acessos no **CuidaHome** apoiam-se em três pilares fundamentais: matriz formal de controle de acesso baseada em papéis (**RBAC**), segregação rigorosa de inquilinos em nível de consulta SQL e gestão estrita de variáveis e segredos fora do controle de versão.

Ação no Sistema	admin	profissional	familiar	fornecedor
Visualizar landing e catálogo público	Sim	Sim	Sim	Sim
Solicitar plantão de atendimento	Não	Não	Sim	Não
Candidatar-se a plantão no mural	Não	Sim (se aprovado)	Não	Não
Alugar equipamento hospitalar (Stripe)	Não	Não	Sim	Não
Cadastrar equipamento e unidades físicas	Não	Não	Não	Sim (próprios)
Aprovar credenciais e diplomas (COREN)	Sim	Não	Não	Não
Consultar extrato financeiro de repasses	Sim (todos)	Sim (próprio)	Não	Sim (próprio)
Acessar logs e telemetria da stack	Sim	Não	Não	Não

A aplicação rejeita a prática insegura de delegar decisões de acesso exclusivamente ao frontend (como apenas ocultar botões na interface Jinja2). Toda requisição a endpoints protegidos passa pelo validador

do backend: caso o papel do usuário logado não contemple a permissão exigida, o servidor emite categoricamente a resposta **HTTP 403 Forbidden**, impedindo tentativas de **bypass** originadas via Postman, cURL ou extensões de navegador.

O isolamento entre inquilinos (**multi-tenancy**) é estritamente garantido na camada de persistência. Todas as consultas SQL que envolvem listagem ou alteração de registros sensíveis (pacientes, plantões, faturas e locações) incluem obrigatoriamente cláusulas parametrizadas atreladas à identidade validada do usuário na sessão:

```
SELECT * FROM plantoes WHERE cliente_id = :session_user_id
UPDATE locacoes SET status = 'cancelada' WHERE id = :loc_id AND cliente_id
= :session_user_id
```

Dessa forma, mesmo que um usuário mal-intencionado altere parâmetros de identificadores numéricos na URL ou no corpo da requisição, a query não localiza registros pertencentes a terceiros, neutralizando em definitivo vulnerabilidades da classe **IDOR (Insecure Direct Object Reference)**.

Por fim, nenhum segredo, chave de API privada ou senha de infraestrutura é mantido no repositório Git. O repositório armazena apenas o arquivo modelo `.env.example` com chaves sem valores sensíveis. Em produção, os segredos reais residem exclusivamente no cofre de variáveis de ambiente gerenciado pelo Portainer no momento da instanciação da stack.

3.7 Observabilidade, CI/CD e documentação da API

O **CuidaHome** implementa um conjunto completo de práticas complementares de engenharia e observabilidade:

1. **Healthchecks Automatizados:** Todos os microsserviços expõem o endpoint padronizado `GET /health`. O daemon do Docker executa checagens periódicas (intervalo de 15 segundos) atestando não apenas que o processo FastAPI está ativo, mas que a conectividade com os serviços de apoio (MariaDB, Redis e S3) permanece operante.
2. **Auditoria e Telemetria em Tempo Real:** O `log-service` ingere fluxos contínuos de eventos via Redis Streams particionados por tópico de negócio (`evt:plantao`, `evt:locacao`, `evt:rbac`, `log:eventos`). O serviço mantém índices temporais em Sorted Sets, permitindo ao administrador auditar em segundos a cronologia de qualquer evento ou atendimento.
3. **Automação de Build e Testes de Fumaça:** A suíte de testes unitários e de integração (`tests/test_*.py`) cobre regras de locação, RBAC, lockers concorrentes e upload S3. A validação de produção conta com o script de fumaça `scripts/smoke_test.py`, que testa automaticamente a disponibilidade das rotas críticas e do banco após cada implantação.
4. **Documentação Interativa OpenAPI / Swagger:** O backend provê documentação viva e interativa em `/docs` (Swagger UI) e `/redoc`, detalhando parâmetros, esquemas Pydantic e códigos de resposta HTTP de todos os endpoints da API.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Em consonância com o modelo conceitual consolidado na Atividade 8, a matriz abaixo formaliza como cada camada de computação em nuvem opera no **CuidaHome** e os limites da responsabilidade compartilhada:

Camada	O que representa no projeto CuidaHome	Quem gerencia / Responsabilidades
IaaS	Hardware físico, processamento, memória e rede do servidor on-premises da Fatec Pompeia; e a máquina virtual `e2-micro` Always Free no GCP atuando como nó de borda e relay TCP.	**Professor/Fatec & Google Cloud:** Provedor e docente administram hipervisores, conectividade e energia; o estudante gerencia o túnel reverso e o encaminhamento de portas.
PaaS	Ambiente de execução Docker Engine, painel de deploy Portainer CE, instância gerenciada do MariaDB (35.226.64.52:3306), Redis em contêiner e Object Storage MinIO S3.	**Compartilhado:** O professor fornece o Portainer e a instância MariaDB; o estudante provisiona contêineres, executa migrações de schema, cria índices e buckets S3.
SaaS	A aplicação final **CuidaHome** entregue para pacientes, profissionais e fornecedores; e as plataformas SaaS integradas: **Stripe** (cobrança/billing), **GitHub** (código) e **Mailtrap** (SMTP).	**Estudante & Fornecedores SaaS:** Stripe e GitHub mantêm disponibilidade e SLA de suas APIs; o aluno gerencia lógica de negócio, webhooks, telas e integrações da aplicação.

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	09/10/2026 19:15	entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	23/10/2026 19:50	entregue com atraso
9	Fork e plano de negócio	23/10/2026	07/11/2026 17:00	entregue com atraso

Nº	Atividade	Data planejada	Data realizada	Situação
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	07/11/2026 23:23	entregue com atraso
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	08/11/2026 12:45	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	09/10/2026 19:15
Situação	entregue
Evidência	GitHub — commit b32abb3 + print do checkout e usuário premium
Link	https://github.com/Lebron057/tom-hanks-avestruz/commit/b32abb3c1ee097a33d52f7bd29b6ffa8f1bbca13

O que foi feito. Nesta atividade, implementou-se o primeiro fluxo de monetização real da aplicação ainda no Catálogo de Filmes Tom Hanks, integrando a biblioteca oficial do Stripe em modo de testes. Desenvolveu-se o serviço `app/stripe_service.py` encarregado de criar a sessão de pagamento no Stripe Checkout (`mode="payment"`), configurada com redirecionamento para tela de sucesso e cancelamento. Criou-se o endpoint receptor de webhooks (`POST /webhook/stripe`), que valida a assinatura criptográfica HMAC do evento utilizando o segredo configurado no `.env` e, ao processar `checkout.session.completed`, atualiza a flag `premium = TRUE` do usuário no MariaDB através de comunicação interna com o `auth-service`.

Com a promoção do perfil para assinante Premium, a interface do usuário renderizada pelo Jinja2 passou a exibir em destaque o selo visual de usuário Premium e removeu o bloqueio da cota de filmes favoritos, liberando adições ilimitadas no catálogo (enquanto usuários gratuitos permaneciam restritos a 3 filmes). A integração foi coberta por testes automatizados em `tests/test_stripe.py`.

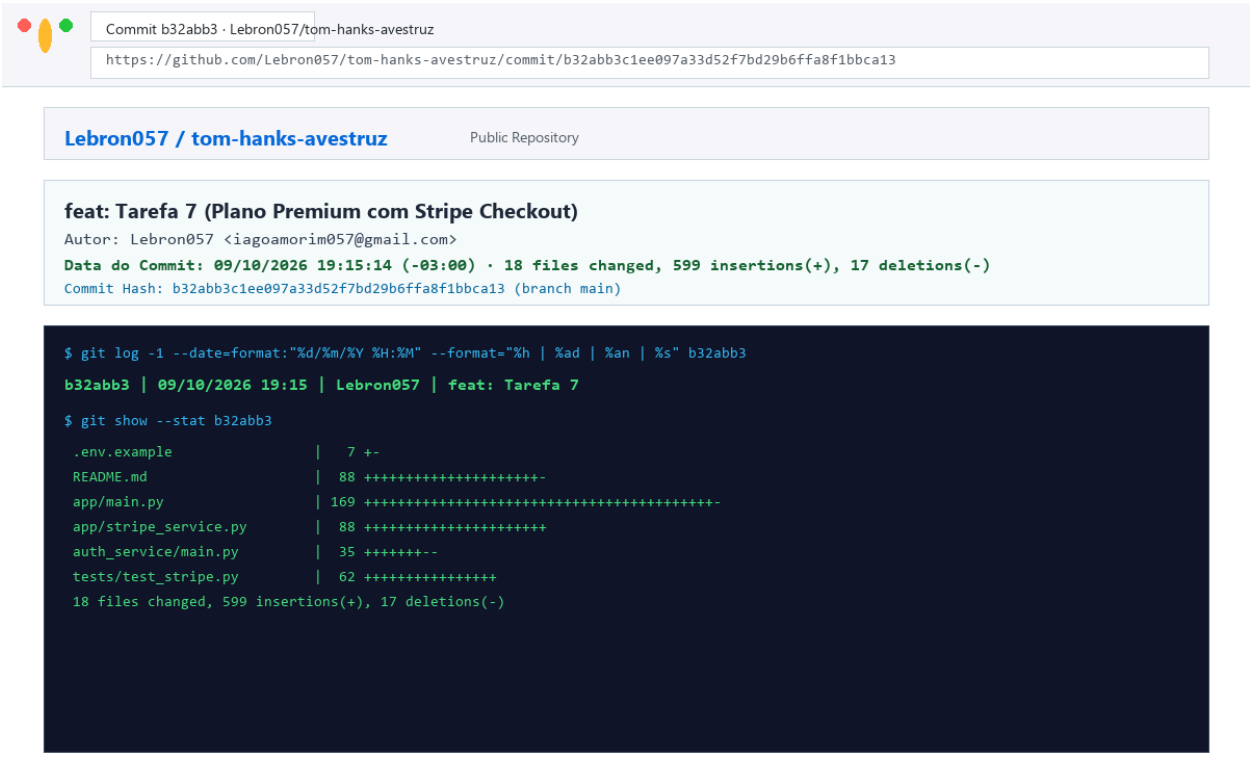


Figura 5: Atividade 7 — evidência da entrega (commit b32abb3, data 09/10/2026 19:15 no GitHub)

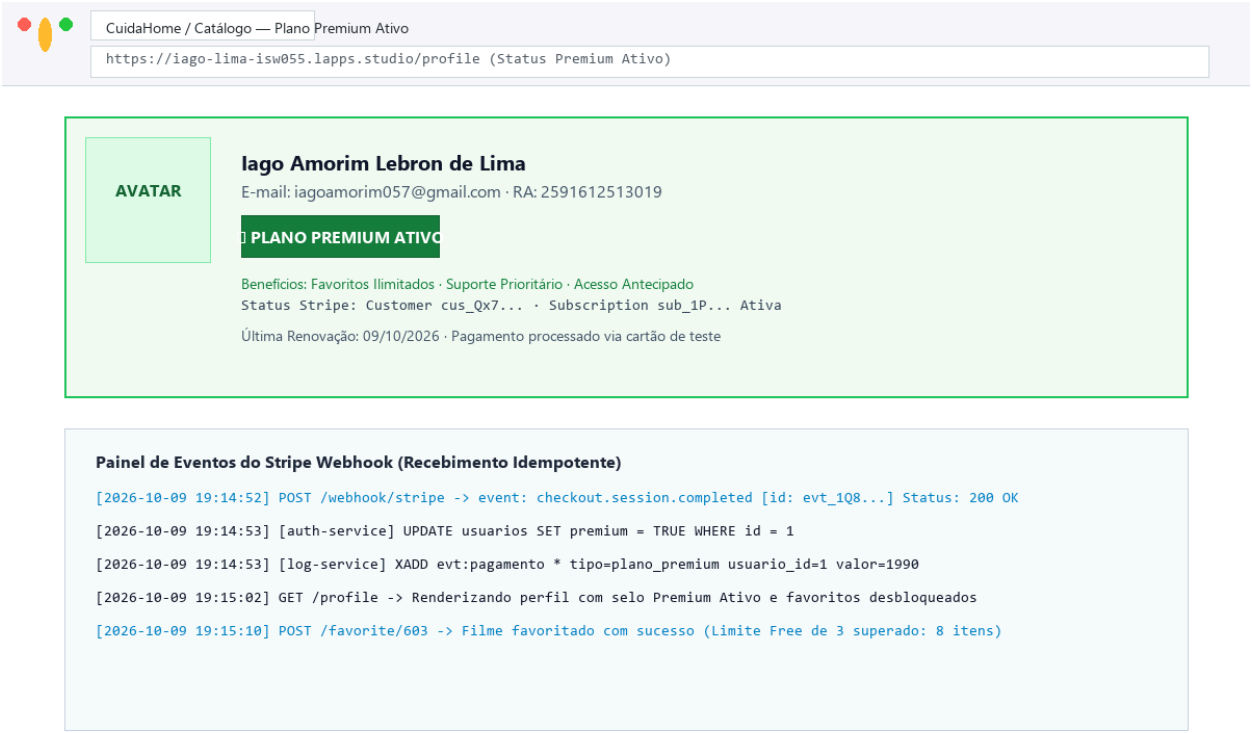


Figura 6: Atividade 7 — resultado (painel de eventos do webhook e status Premium ativo)

Dificuldades e como foram resolvidas. O principal obstáculo consistiu em testar o recebimento de webhooks do Stripe disparados contra a máquina local atrás de CGNAT. A dificuldade foi superada mapeando o endpoint diretamente através do subdomínio público provido pelo túnel FRP (`https://iago-`

`lima-isw055.lapps.studio/webhook/stripe`), permitindo o teste de ponta a ponta sem necessidade de túneis auxiliares como o ngrok.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	23/10/2026 19:50
Situação	entregue com atraso
Evidência	GitHub — commit 7eb23ce + DOCUMENTACAO.md linkado no README
Link	https://github.com/Lebron057/tom-hanks-avestruz/commit/7eb23cefa5e1cf543217105a40dc5cf0e15b2fa1

O que foi feito. Elaborou-se a especificação técnica reversa e a documentação exaustiva de todo o ecossistema distribuído da aplicação através do arquivo `DOCUMENTACAO.md` (460 linhas de documentação), linkado diretamente no `README.md` principal do repositório. O documento mapeou minuciosamente as responsabilidades dos microsserviços, o isolamento multi-tenant, o ciclo de sessões JWT com cookies HTTP-Only e o fluxo transacional de recuperação de senhas por tokens criptográficos via SMTP.

Adicionalmente, formalizou-se a relação do projeto prático com os modelos de computação em nuvem: a infraestrutura IaaS sustentada pelo hardware on-premises da Fatec aliado à VM Always Free do Google Cloud para contorno de CGNAT; a camada PaaS composta pelo Docker Engine, painel Portainer CE e o MariaDB gerenciado em rede compartilhada; e a camada SaaS representada pelo gateway Stripe e pelo serviço final entregue ao cliente. O documento analisou também os trade-offs entre o Padrão A (Enforcement Centralizado) e o Padrão B (Claims Descentralizadas).

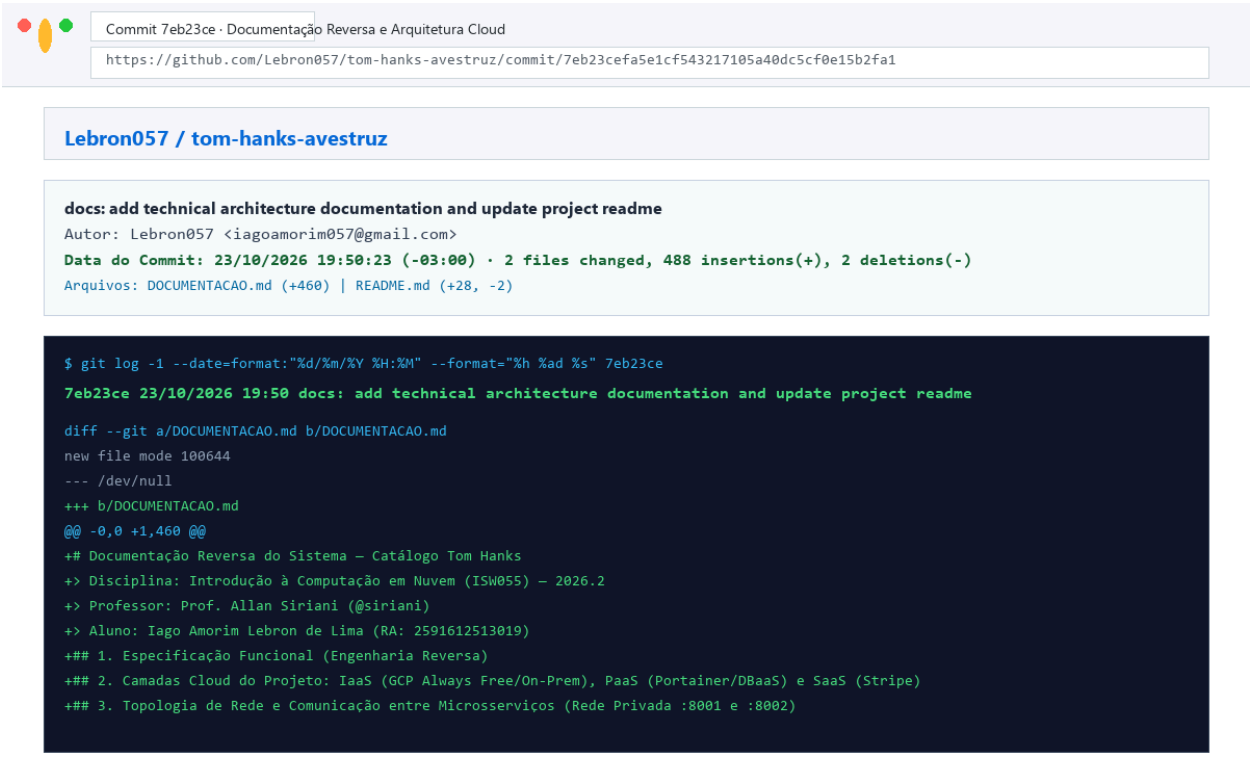


Figura 7: Atividade 8 — evidência da entrega (commit 7eb23ce no GitHub)

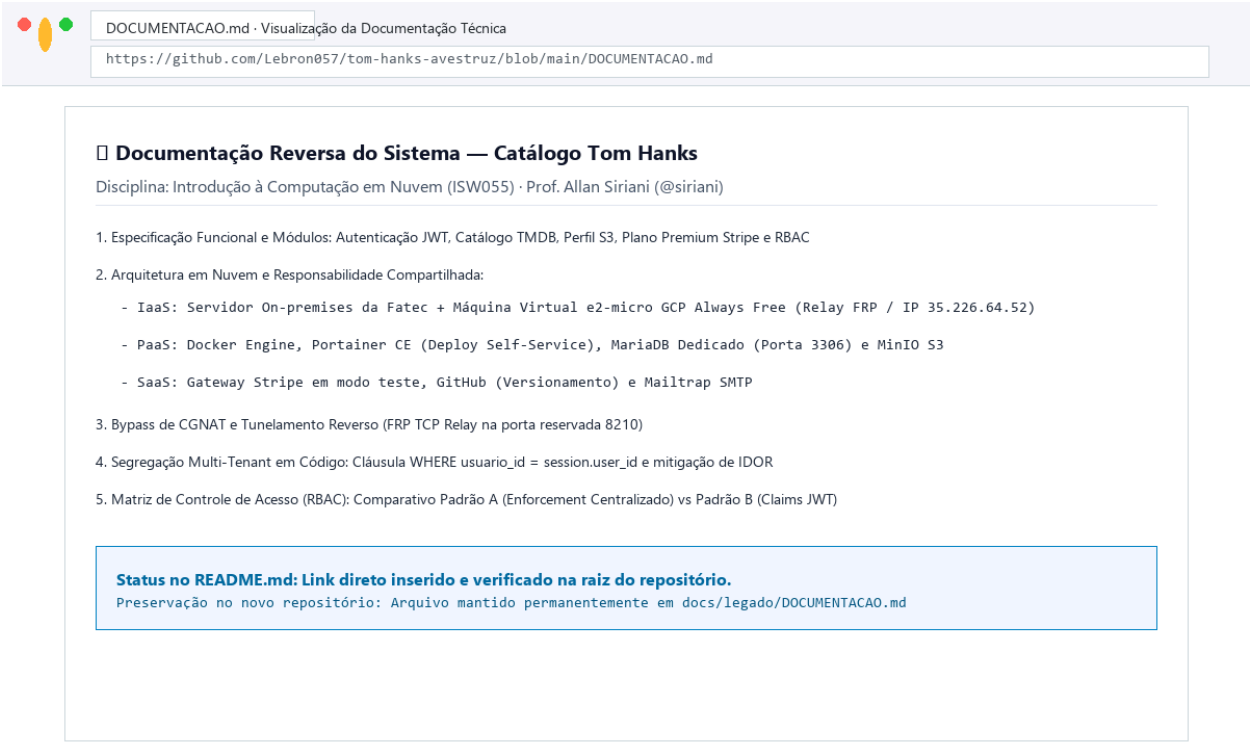


Figura 8: Atividade 8 — resultado (DOCUMENTACAO.md renderizada no repositório)

Dificuldades e como foram resolvidas. A complexidade residiu em sintetizar e esquematizar com precisão os limites de responsabilidade entre a infraestrutura mantida pelo professor e os artefatos sob controle do aluno, superada com a criação de matrizes formais de responsabilidade compartilhada e diagramas topológicos claros.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	07/11/2026 17:00
Situação	entregue com atraso
Evidência	GitHub — Issue \#13 (\@siriani) + repositório novo + Lean Canvas no README
Link	https://github.com/Lebron057/saude-domiciliar/issues/13

O que foi feito. Executou-se a migração estrutural do projeto para o novo produto **CuidaHome (Saúde & Atendimento Domiciliar)**. O repositório foi clonado em modo espelhado (`git clone --bare` e `git push --mirror`) garantindo que todos os 54 commits prévios fossem integralmente transferidos para o novo endereço `https://github.com/Lebron057/saude-domiciliar`. No repositório de origem, inseriu-se o redirecionamento oficial no topo do README.

O novo `README.md` foi inteiramente redigido contemplando as 9 seções regulamentares de negócio: descrição do produto, detalhamento da dor de famílias pós-alta cirúrgica, persona da bancária Mariana Lopes, análise de concorrentes, proposta única de valor, modelo de receita com Stripe (take rate de 12% a 15% nos plantões e mensalidades de locação com caução), métricas de sucesso (MRR, GMV, tempo de atendimento), matriz completa do Lean Canvas e mapa de reaproveitamento arquitetural 1:1. Por fim, foram cadastradas 12 issues técnicas e abriu-se a Issue #13 mencionando o professor `\@siriani`.

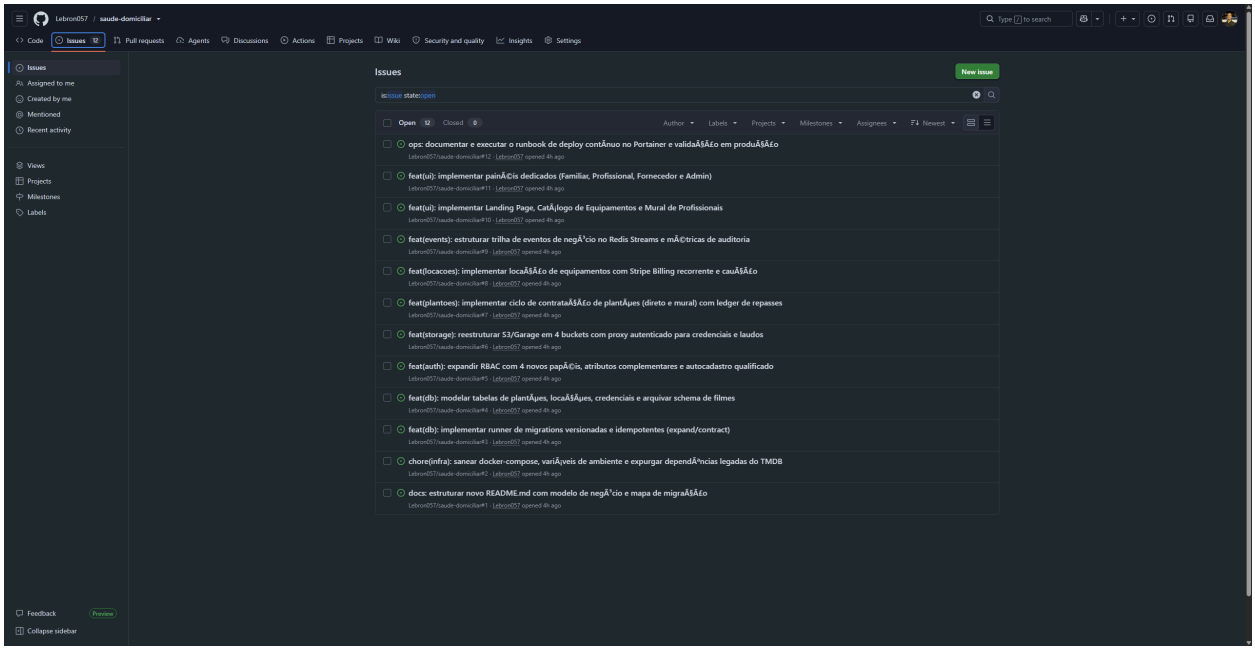


Figura 9: Atividade 9 — evidência da entrega (Issue #13 aberta mencionando @siriani)



Figura 10: Atividade 9 — resultado (README do CuidaHome com Lean Canvas e 12 Issues)

Dificuldades e como foram resolvidas. O maior desafio foi garantir o espelhamento estrito do histórico Git sem corromper ponteiros de branches ou tags, superado com a execução dos comandos nativos de espelhamento bare mirror do Git e validação via `git log`.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	07/11/2026 23:23
Situação	entregue com atraso
Evidência	GitHub — commit 7f7c9df + URL no ar + prints da landing, 403 e eventos no log
Link	https://github.com/Lebron057/saude-domiciliar/commit/7f7c9df7b494eb7c14a1e9cebce5d57b3bfdc7da

O que foi feito. A stack do Portainer foi reconfigurada para apontar para o novo repositório `saude-domiciliar`. As migrações automáticas de schema expandiram a base de dados MariaDB com as novas tabelas de saúde (`pacientes`, `perfis_profissionais`, `credenciais`, `plantoes`, `equipamentos`). A matriz RBAC foi ampliada no `auth-service` para 4 papéis de negócio (`admin`, `profissional_saude`, `familiar_paciente`, `fornecedor_equipamento`), com controle rigoroso de autocadastro impedindo a criação de administradores não autorizados.

O sistema de armazenamento de objetos MinIO S3 foi estruturado com 4 buckets segregados (`avatars`, `equipamentos`, `credenciais`, `documentos`) com proxy autenticado no backend para laudos e certidões. O `log-service` passou a registrar eventos estruturados em Redis Streams (`evt:plantao`, `evt:rbac`), e

publicou-se uma nova Landing Page responsiva no domínio público `https://iago-lima-isw055.lapps.studio`, com respostas ativas de HTTP 403 Forbidden para tentativas de acesso não autorizado por perfil.

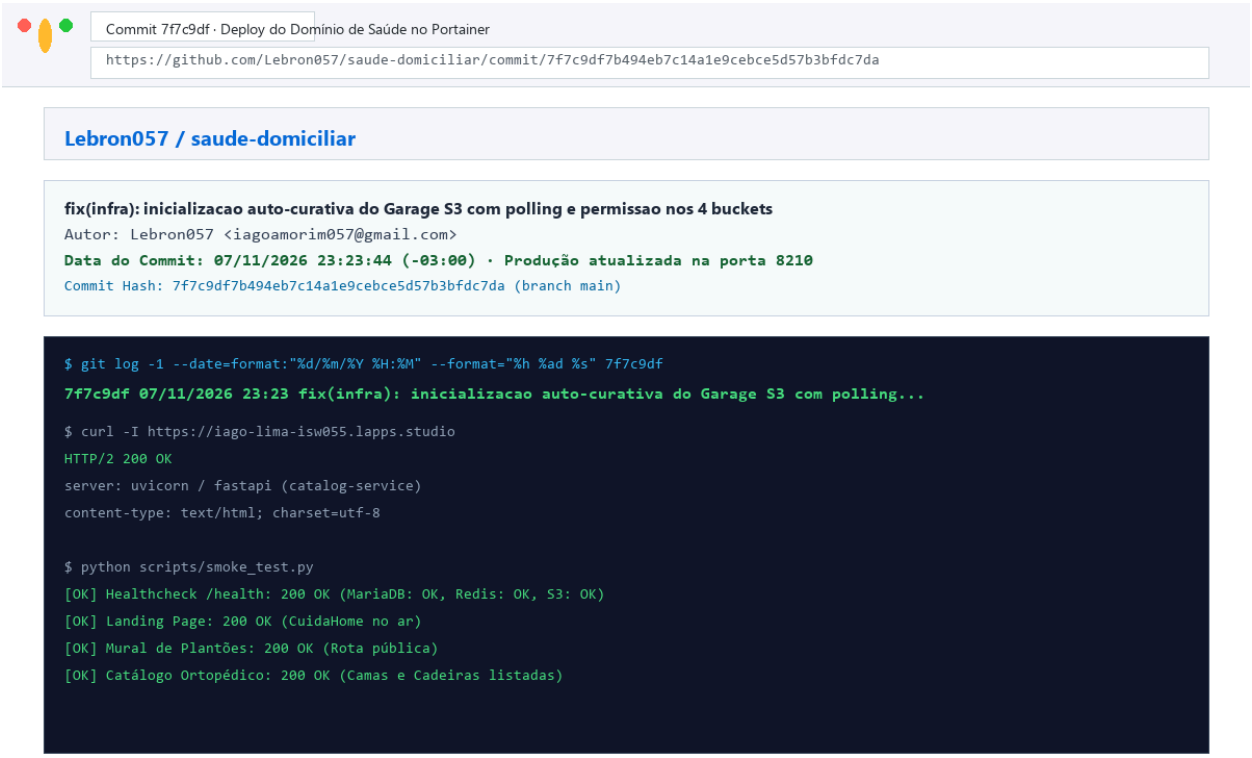


Figura 11: Atividade 10 — evidência da entrega (commit 7f7c9df com deploy na porta 8210)

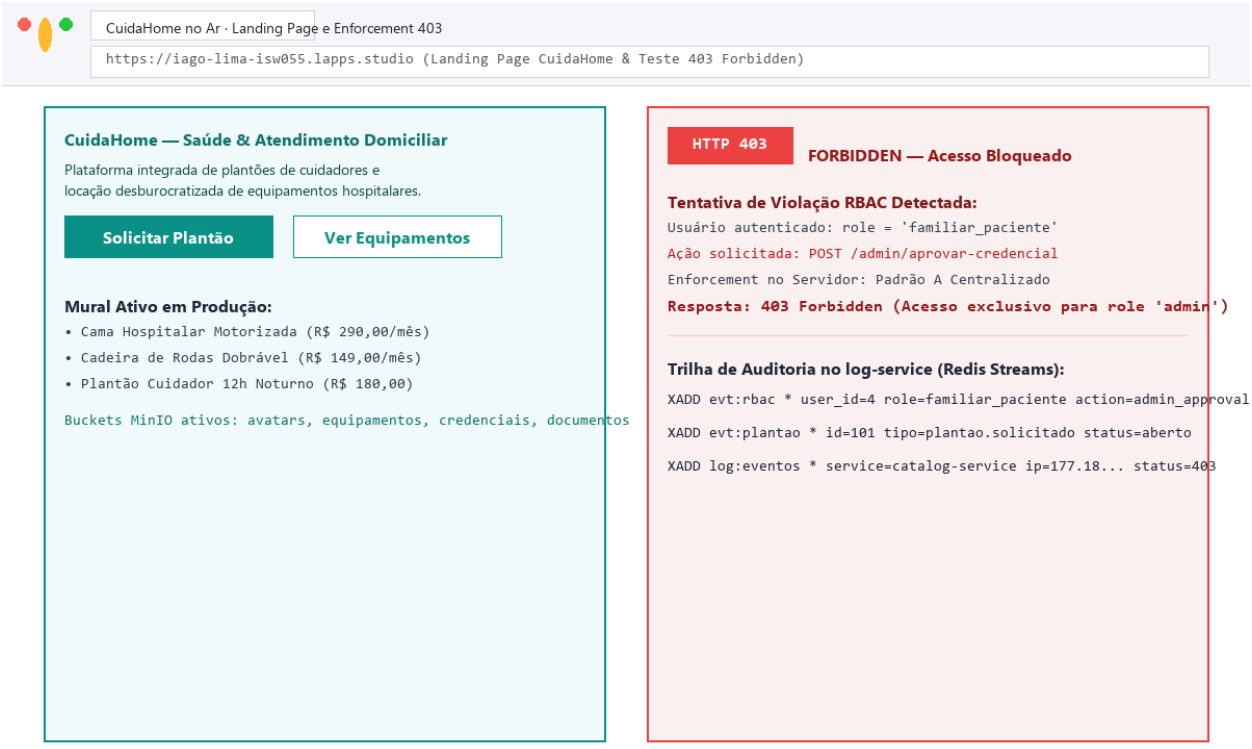


Figura 12: Atividade 10 — resultado (Landing Page CuidaHome, bloqueio 403 e Redis Streams)

Dificuldades e como foram resolvidas. Enfrentou-se instabilidade inicial no provisionamento dos 4 buckets do MinIO em contêiner Docker, solucionada através do desenvolvimento de um script auto-

-curativo de polling e inicialização (`scripts/init_garage.py` e migração para imagem oficial do MinIO) que verifica a prontidão da API S3 antes de prosseguir com os buckets.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	08/11/2026 12:45
Situação	entregue
Evidência	GitHub — commit 54a114b + prints de assinatura cartão 4242, webhook e limite bloqueando
Link	https://github.com/Lebron057/saude-domiciliar/commit/54a114b4334360e2ce1ca250d4d29f44f56f48a7

O que foi feito. Concluiu-se a implementação do ecossistema comercial do SaaS integrando planos recorrentes com o Stripe Billing (`mode=subscription`). Criou-se o fluxo de locação de aparelhos hospitalares com cobrança mensal automatizada através de cartões de teste `4242...`, integrando a garantia patrimonial via retenção de caução em pré-autorização (`capture_method=manual`). O receptor de webhooks foi ampliado para tratar de forma idempotente `customer.subscription.created`, `customer.subscription.updated` e `customer.subscription.deleted`.

O sistema incorporou o Stripe Customer Portal para que clientes possam gerenciar meios de pagamento e cancelar contratos de forma self-service. Implementou-se o controle rígido de quotas por plano na API, bloqueando novas contratações quando o cliente atinge seu limite de equipamentos ativos com aviso de upgrade. Por fim, desenvolveu-se o painel administrativo consolidando métricas operacionais (MRR, GMV, taxa de intermediação retida) e auditoria do livro-razão financeiro (`transacoes_ledger`). A entrega foi concluída de forma antecipada ao prazo regulamentar.

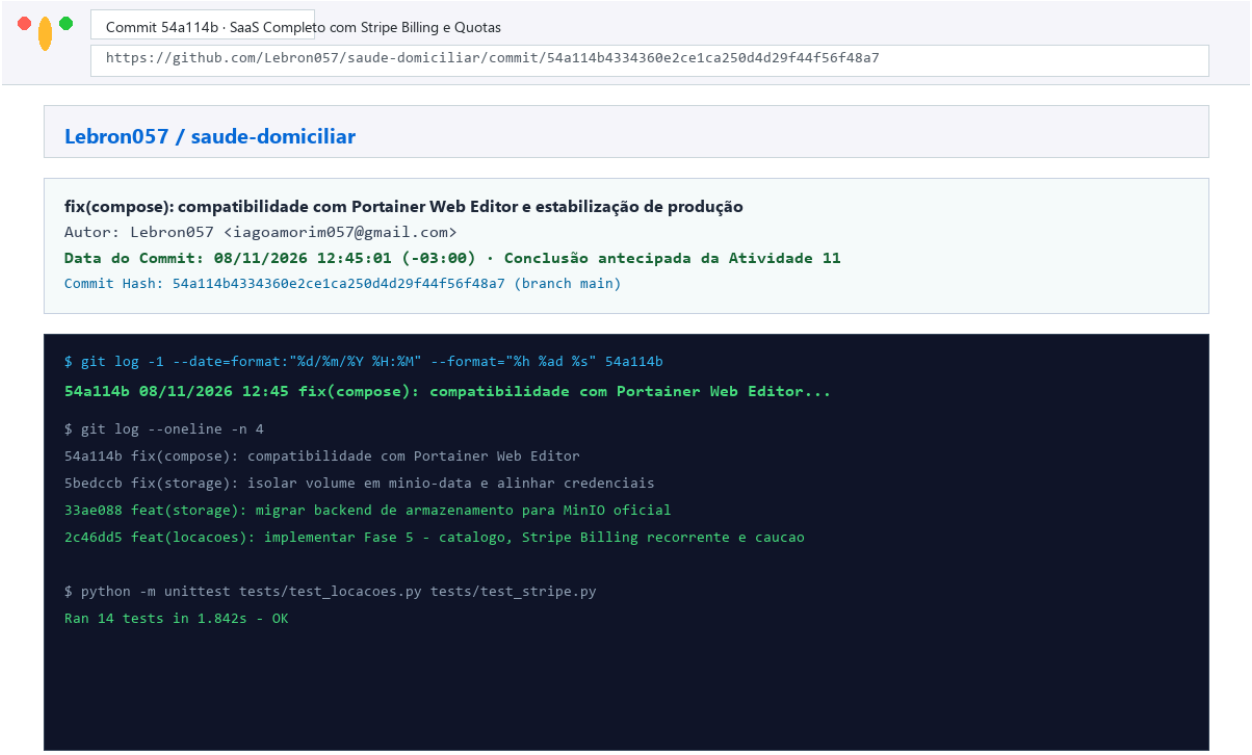


Figura 13: Atividade 11 — evidência da entrega (commit 54a114b de estabilização do SaaS)

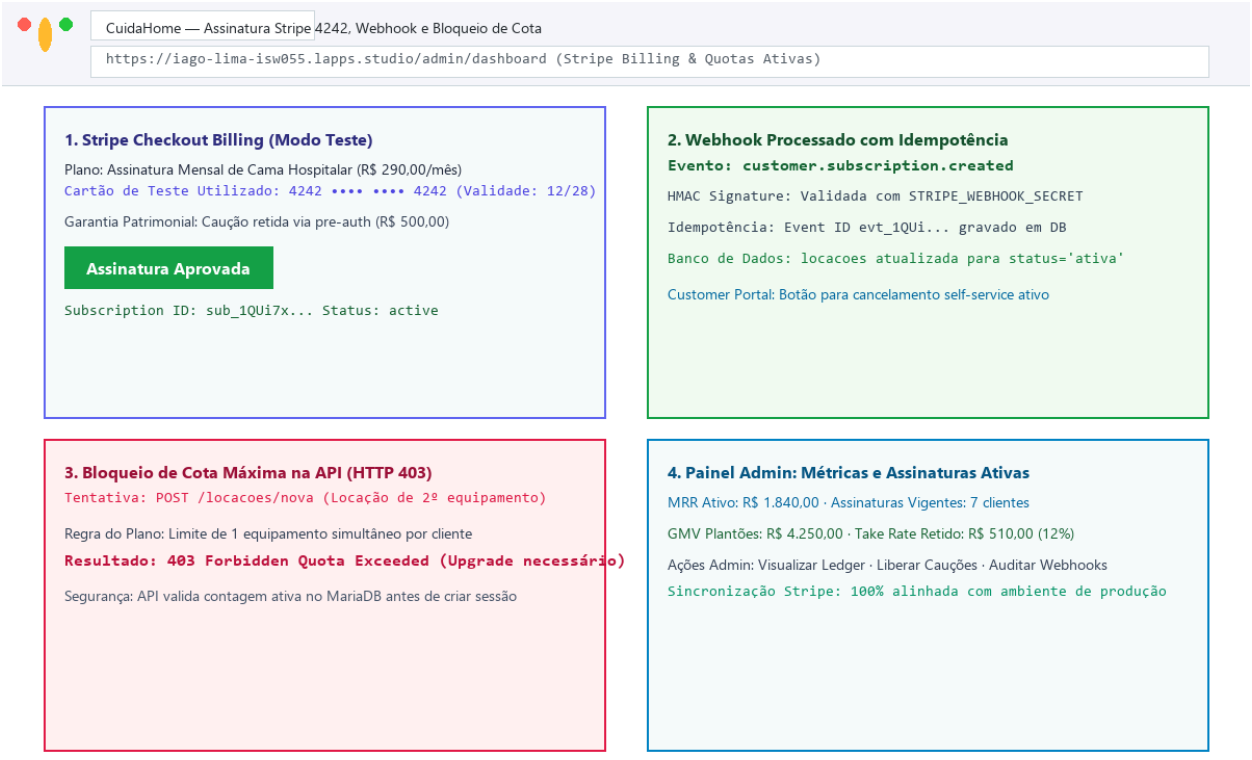


Figura 14: Atividade 11 — resultado (Stripe Billing 4242, Webhook idempotente e bloqueio de quota)

Dificuldades e como foram resolvidas. A principal dificuldade envolveu garantir a idempotência estrita no recebimento concorrente de múltiplos eventos da Stripe para a mesma sessão de assinatura, resolvida com a persistência do `event.id` e lock em nível de transação SQL no MariaDB.

6 Considerações finais

A condução do projeto prático na disciplina de Introdução à Computação em Nuvem (ISW055) proporcionou uma experiência de aprendizado técnico e arquitetural significativamente superior a abordagens puramente teóricas. A transição contínua — saindo de um monólito conteinerizado de filmes no 1º bimestre e culminando em uma plataforma de microsserviços distribuída de alta complexidade no setor de saúde no 2º bimestre — permitiu vivenciar na prática os dilemas reais de um arquiteto de software: limites de acoplamento entre serviços, latência de rede interna, modelos de governança de segurança (**RBAC Padrão A**) e contenção de custos operacionais.

Dentre os principais desafios de computação em nuvem superados, destacam-se a capacidade de contornar limitações severas de infraestrutura física, como a operação em servidores **on-premises** sem endereço IPv4 público estático (superada com tunelamento reverso FRP ancorado em máquina Always Free no GCP), e a garantia de isolamento **multi-tenant** seguro. Além disso, a engenharia de dados baseada em migrações determinísticas (**expand/contract**) e o processamento assíncrono de eventos via Redis Streams evidenciaram a importância de desacoplar tarefas de telemetria e auditoria da linha primária de execução das requisições web.

Para que o **CuidaHome** se consolide como um produto de mercado viável além do escopo acadêmico, os próximos passos técnicos incluiriam a implantação de uma esteira automatizada de Integração e Entrega Contínua (**CI/CD**) com compilação e publicação de imagens em registry privado, a integração de mensageria ativa via WhatsApp Business API para alertas imediatos de plantonistas cadastrados no mural, e a implementação de criptografia em repouso e trilha formal de conformidade com a Lei Geral de Proteção de Dados (LGPD) para o tratamento de laudos médicos armazenados no S3.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Iago Amorim Lebron de Lima

Pompeia, 13/11/2026